

CSCE 735 PARALLEL COMPUTING · MODULE 01 · LECTURE NOTES

Why Parallel, and What It Costs

The free lunch ended, and the bill is restructuring.

MODULE OUTCOMES

Explain why single-thread performance stopped improving.

State Amdahl's law and compute its consequences.

State Gustafson's law and say when it applies instead.

Distinguish strong from weak scaling.

Measure and report speedup honestly.

1 Why everything became parallel

DENNARD SCALING ENDED; MOORE'S LAW DID NOT

These are routinely conflated and they are different observations with different fates.

Moore's law is the observation that transistor density doubles roughly every two years. It continued for well over a decade after the change described here, and is slowing now for reasons of physics and economics.

Dennard scaling is the observation that as transistors shrink, power density stays constant — smaller transistors need less voltage and switch faster, so you get more, faster transistors for the same power. **This broke around 2005**, because leakage current stopped falling as voltage dropped, so reducing voltage further stopped saving power.

So clock speeds plateaued while transistor budgets kept growing. The transistors still arrived and could no longer be spent on making one core faster. **They went into more cores, wider vector units, larger caches, and specialised accelerators** — every one of which requires the programmer to restructure the computation to benefit. **That restructuring is the bill**, and this course is about paying it.

Resource	Trend	What it requires of you
Clock frequency	Flat since about 2005.	Nothing. It was free, and it is over.
Core count	Still growing: 64 to 128 cores is now ordinary in a server.	Explicit parallelism — threads, tasks, processes (Modules 03, 06, 10).
SIMD width	Growing: 128 bits, then 256, then 512.	Vectorisable loops — regular access patterns, no dependencies, no unpredictable branches (Module 02).

Resource	Trend	What it requires of you
Cache capacity	Growing slowly.	Locality. The algorithm must reuse data while it is still resident (CSCE 614).
Specialised units	Growing fastest — tensor cores, matrix engines, media accelerators.	Using the right library , because hand-writing for them is rarely worthwhile.

Every row after the first requires the programmer to do something, and that is the whole reason this course exists. Before 2005 a program got faster by waiting; now it gets faster by being rewritten.

2 Amdahl's law

$$S(n) = 1 / (s + (1 - s)/n) \quad S(\infty) = 1/s$$

where s is the fraction of the total work that is inherently serial. **The bound depends only on s , not on the hardware** — which is why reducing the serial fraction matters far more than acquiring more cores.

Serial fraction s	S at $n=16$	S at $n=256$	S at $n=1024$	Ceiling
0.1%	15.9	203	506	1000
1%	13.9	72	91	100
5%	9.1	17.4	19.6	20
10%	6.4	9.7	9.9	10
50%	1.9	2.0	2.0	2

Look at the 5% row. Going from 16 cores to 1024 — sixty-four times the hardware — takes the speedup from 9.1 to 19.6. **A factor of about two, for sixty-four times the machine.** This single row explains why large core counts are not a general-purpose solution and why so much effort goes into shrinking s .

THE SERIAL FRACTION IS LARGER THAN IT LOOKS

It is not only the obviously sequential code. Everything that does not scale with the processor count lands in s , whether or not it looks serial.

Startup and teardown. Reading the input file, allocating memory, building the data structure, writing the output. These often run on one thread and often take longer than anyone estimated.

Synchronisation. Time spent waiting at a barrier is time not spent computing, and a barrier costs the *slowest* thread's time multiplied by the thread count.

Load imbalance. If one thread's share takes twice as long as the others', every other thread's idle time is effectively serial (Module 06).

And the parallel overhead itself — thread creation, task scheduling, communication, false sharing. **All of it lands in s** , which is why measured speedups fall short of Amdahl's prediction rather than matching it.

3 Gustafson's law

PROBLEMS GROW WITH THE MACHINE

Amdahl assumes a fixed problem size and asks how much faster a larger machine can solve it. Under that assumption the answer is discouraging, and the law is frequently cited as a reason parallelism is futile.

Gustafson observed that the assumption is usually wrong. Nobody acquires a machine with a hundred thousand cores in order to solve the same problem they were solving before, slightly faster. They solve a larger problem — a finer grid, more particles, a longer simulation, a bigger model.

And the serial portion usually does not grow with the problem size. Parsing a configuration file, initialising, and writing a summary cost roughly the same for a grid of any resolution.

So the scaled speedup — work done per unit time as both the machine and the problem grow — stays close to linear. This is why machines with enormous core counts are worth building, and it is not in conflict with Amdahl: the two laws answer different questions about different situations.

	Strong scaling (Amdahl)	Weak scaling (Gustafson)
Held fixed	The total problem size.	The work per processor.
As n grows	Each processor gets a smaller share.	The total problem grows proportionally.
Bounded by	The serial fraction — hard ceiling at 1/s.	Can remain near linear.
Communication	Worsens — less computation per processor against roughly the same communication, so the ratio degrades.	Stays roughly constant, since both grow together.
Answers	'How much faster can I solve <i>this</i> problem?'	'How large a problem can I solve in a fixed time?'

REPORT BOTH, AND STATE THE BASELINE

Weak scaling curves look better, nearly always. A result showing only weak scaling is making a substantially weaker claim than a casual reader will take it to be.

Strong scaling is what most users actually care about. They have a problem of a particular size and want the answer sooner; the option of solving a bigger problem instead is not useful to them.

So measure and report both, clearly labelled. The pair tells a complete story and either alone does not.

And state the serial baseline explicitly. Speedup measured against an unoptimised serial implementation is the oldest trick in performance reporting and remains common — a poorly written serial version makes any parallel version look excellent, and the comparison measures the quality of the baseline rather than the parallelisation.

- **Speedup against an unoptimised baseline.** Parallelising badly written serial code produces flattering numbers that mean nothing. **Project 1 requires a profiled, optimised serial baseline for exactly this reason.**

- **Speedup against a different algorithm.** Comparing a parallel implementation of a good algorithm against a serial implementation of a worse one.
- **Scaling plots that stop exactly where the curve would have bent.** If a plot ends at 16 cores on a 64-core machine, ask why.
- **Superlinear speedup presented as a triumph.** It is *possible* — n processors have n times the cache, so a working set that did not fit on one may fit when divided. But it usually indicates that the serial baseline was cache-thrashing, which means the baseline was poorly chosen. **It deserves an explanation, not a celebration.**
- **Best-of-many timings rather than a distribution.** Report the median and the spread; a minimum over twenty runs describes a machine with no other load, which is not the machine anyone has.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Stanford CS149 / CMU 15-418 — Lecture 1 and the scaling lectures (free)	Why parallelism, Amdahl, and the architectural history. The primary source for this module.
Sutter — The Free Lunch Is Over (2005, free)	The essay that named the change in §1, written as it was happening. Short and still accurate.
Gustafson — Reevaluating Amdahl's Law (1988, free)	Two pages, and it reframes the whole question. Read it immediately after Amdahl.
Hoefler & Belli — Scientific Benchmarking of Parallel Computing Systems (free)	How to measure and report honestly — the material in §3, made rigorous. Read it before producing any scaling plot.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Compute Amdahl speedup for $s = 0.001, 0.01, 0.05, 0.1$ at $n = 2, 16, 256, 1024$. Plot the curves on one figure.
- 2 For a program you have written, estimate the serial fraction by timing the phases. Include startup and I/O.
- 3 Instrument that program to measure time spent at barriers and add it to your serial fraction estimate.
- 4 Compute the Gustafson scaled speedup for the same serial fractions and compare against Amdahl on one plot.

- 5 Take any parallel program and measure both strong and weak scaling. Plot them separately and identify where each departs from ideal.
- 6 **Deliberately construct a misleading speedup claim** using an unoptimised baseline, then correct it. Report both numbers.
- 7 Construct a case exhibiting superlinear speedup from cache effects, and explain it.
- 8 Run the same benchmark twenty times and plot the distribution of timings. Report median and interquartile range rather than the minimum.
- 9 Find a published parallel computing result and check it against the five failure modes in §3.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish Moore's law from Dennard scaling and say which ended when.
- 2 Name five resources transistors went into and what each requires of the programmer.
- 3 State Amdahl's law and compute the speedup for $s = 0.05$ at $n = 1024$.
- 4 Give five things that count toward the serial fraction.
- 5 State Gustafson's law and explain why it does not contradict Amdahl.
- 6 Distinguish strong from weak scaling on five axes.
- 7 Why do weak scaling curves look better, and what should you report?
- 8 Give five ways speedup is misreported.
- 9 Why does superlinear speedup require explanation?